

هندسة البرمجيات (Software Engineering)

الفصل الخامس: نمذجة البرمجيات

(Software Modeling)

المخطط العام – Outline

- النموذج (Model)
- النموذج الرشيق (Agile Model)
- لغة النمذجة الموحدة (UML)
- حالات الاستخدام (Use Case)
- مخطط الصنف (Class Diagram)
- مسؤوليات وتعاون الأصناف (CRC)
- مخطط التسلسل (Sequence Diagram)

النموذج (Model)

- النموذج (Model): هو مفهوم يصف جانباً أو أكثر من جوانب مشكلة ما أو حلاً محتملاً لمعالجة هذه المشكلة.
- تقليدياً، يُنظر إلى النماذج على أنها تحتوي على رسم بياني واحد أو أكثر، بالإضافة إلى أي توثيق مصاحب.

!؟ لماذا نقوم بعملية النمذجة؟

هناك سببان رئيسيان:

١. لفهم ما الذي يتم بناؤه.
٢. لتسهيل جهود التواصل داخل الفريق أو مع أصحاب المصلحة في المشروع.

النموذج الرشيق (Agile Model)

- من وجهة نظر البساطة، يمكن القول إن تطوير الأنظمة لا يحتاج أكثر من: البرمجة (Coding)، والاختبار (Testing)، والاستماع (Listening).
- إذا أنجزت هذه الأنشطة بشكل جيد، فإن النتيجة يجب أن تكون نظامًا يعمل.
- لكن في الواقع، هذا لا يكفي. فيمكنك التقدم لمسافة جيدة دون تصميم، لكنك ستتعثّر في نقطة معينة.
- إذ يصبح النظام معقدًا للغاية، وتنعدم وضوحية التبعيات بداخله.

لتجنّب ذلك:



- يجب إنشاء هيكل تصميمي (أي نموذج-Model) ينظّم المنطق داخل النظام.
- التصميم الجيد يقلل التبعيات، مما يعني أن تعديل جزء من النظام لن يؤثر على الأجزاء الأخرى.

نموذج البرمجة المتطرفة (XP Model)

- النمذجة (Modeling) والتوثيق (Documentation) هما عنصران مهمان في XP، كما هو الحال في أي منهجية تطوير برمجيات أخرى.
- لكن XP توصي بتقليل الجهد المبذول في النمذجة ليكون فقط بالقدر الكافي.
- تُستخدم النمذجة في XP بشكل أساسي في تصميم البرمجيات.
- تُطوّر مخططات صغيرة وبسيطة تركز على جانب محدد مثل دورة حياة صنف أو التنقل بين الشاشات.
- يمكن استخدام مخططات UML لهذا الغرض.

التصميم البسيط (Simple Design)

- يجب أن يُصمّم النظام بأبسط شكل ممكن في أي لحظة زمنية. ويتم إزالة أي تعقيد إضافي بمجرد اكتشافه.
- لا يُضيع الفريق وقته في التفكير بالمتطلبات المستقبلية.
- لا يتم بناء بنى تحتية حالية لميزات قد تكون مطلوبة لاحقًا.
- التركيز يكون على البنية الحالية وجعلها بأفضل حالة ممكنة.

لغة النمذجة الموحدة (UML)

- ال UML هي المعيار الصناعي لنمذجة الأنظمة الكائنية التوجه (Object-Oriented).
- توفر رموزًا لعدة نماذج تُنتج أثناء تحليل وتصميم الأنظمة الكائنية.
- تُعد UML الآن معيارًا فعليًا (De Facto) في نمذجة الكائنات.

? ما هي UML؟

👉 هي لغة تُستخدم لـ:

- التحديد (Specifying)
- التصور (Visualizing)
- البناء (Constructing)
- التوثيق (Documenting)

لـ عناصر البرمجيات (Software Artifacts)

تقدم لغة النمذجة (UML):

- عناصر النمذجة (Model Elements): المفاهيم والدلالات.
- الترميز (Notation): التمثيل البصري لعناصر النمذجة.
- الإرشادات (Guidelines): نصائح لاستخدام العناصر بالشكل الصحيح.

أنواع نماذج التصميم (Types Design Models)

- تُظهر نماذج التصميم الكائنات (Objects) والأصناف (Classes) والعلاقات (Relationships) بينها.
- النماذج الثابتة (Static Models):
 - تصف الهيكل الثابت للنظام (مثل الأصناف والعلاقات).
- النماذج الديناميكية (Dynamic Models):
 - تصف التفاعلات الديناميكية بين الكائنات.

حالة الاستخدام (Use Case)

حالات الاستخدام تُستخدم لالتقاط متطلبات المستخدم بشكل تفاعلات بين المستخدم (User) والنظام (System).

تعريف (Definition):

- حالة الاستخدام هي وصف عام لمجموعة من التفاعلات بين النظام وأحد المستخدمين أو الأنظمة الأخرى.
- نموذج حالات الاستخدام يساعدنا على التأكد من أننا لم ننس أي وظيفة أساسية، ويسهل علينا رؤية صورة شاملة للنظام.

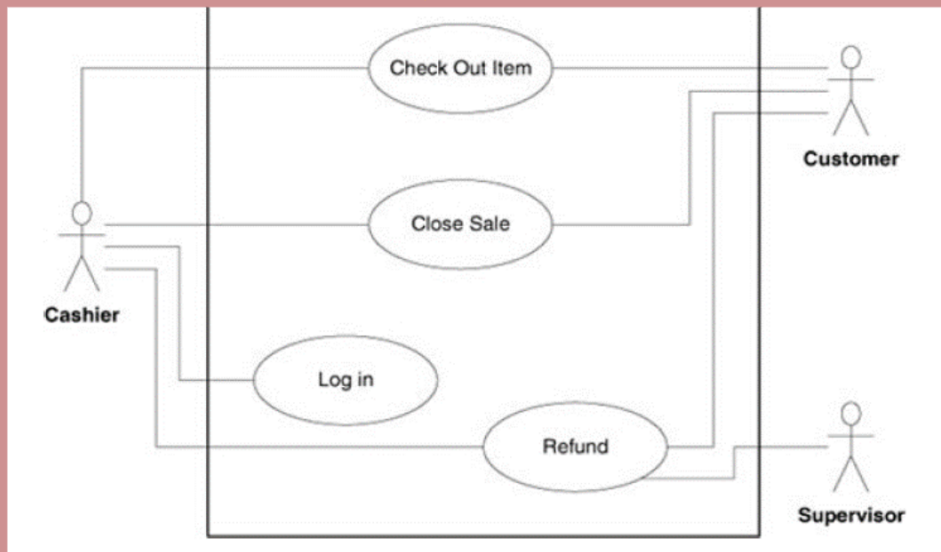
مستويات التفاصيل في حالات الاستخدام:

1. المستوى العالي (High Level): وصف موجز بدون تفاصيل.
2. الموسعة (Expanded): أكثر تفصيلاً مع توضيح كل خطوة في العملية (لا تذكر كيفية استجابة النظام).

خطوات بناء حالات الاستخدام:

- تحديد حدود النظام.
- تحديد الفاعلين (Actors) وحالات الاستخدام (Use Cases).
- رسم مخطط استخدام عالي المستوى (Use Case Diagram).
- كتابة حالات الاستخدام الموسعة للحالات الحرجة أو المعقدة وتأجيل الباقي.

أمثلة على حالات الاستخدام (User Cases Example)



مخطط حدود النظام (Boundary Diagram)

المستطيل الكبير: يمثل حدود النظام،
(كل ما بداخله جزء من النظام قيد التطوير).

خارج المستطيل:

- الفاعلون (Actors) مثل: الكاشير، الزبون، المشرف.
- يمكن أن يكون الفاعل مستخدمًا بشريًا أو نظامًا آخر أو جهازًا.

داخل المستطيل:

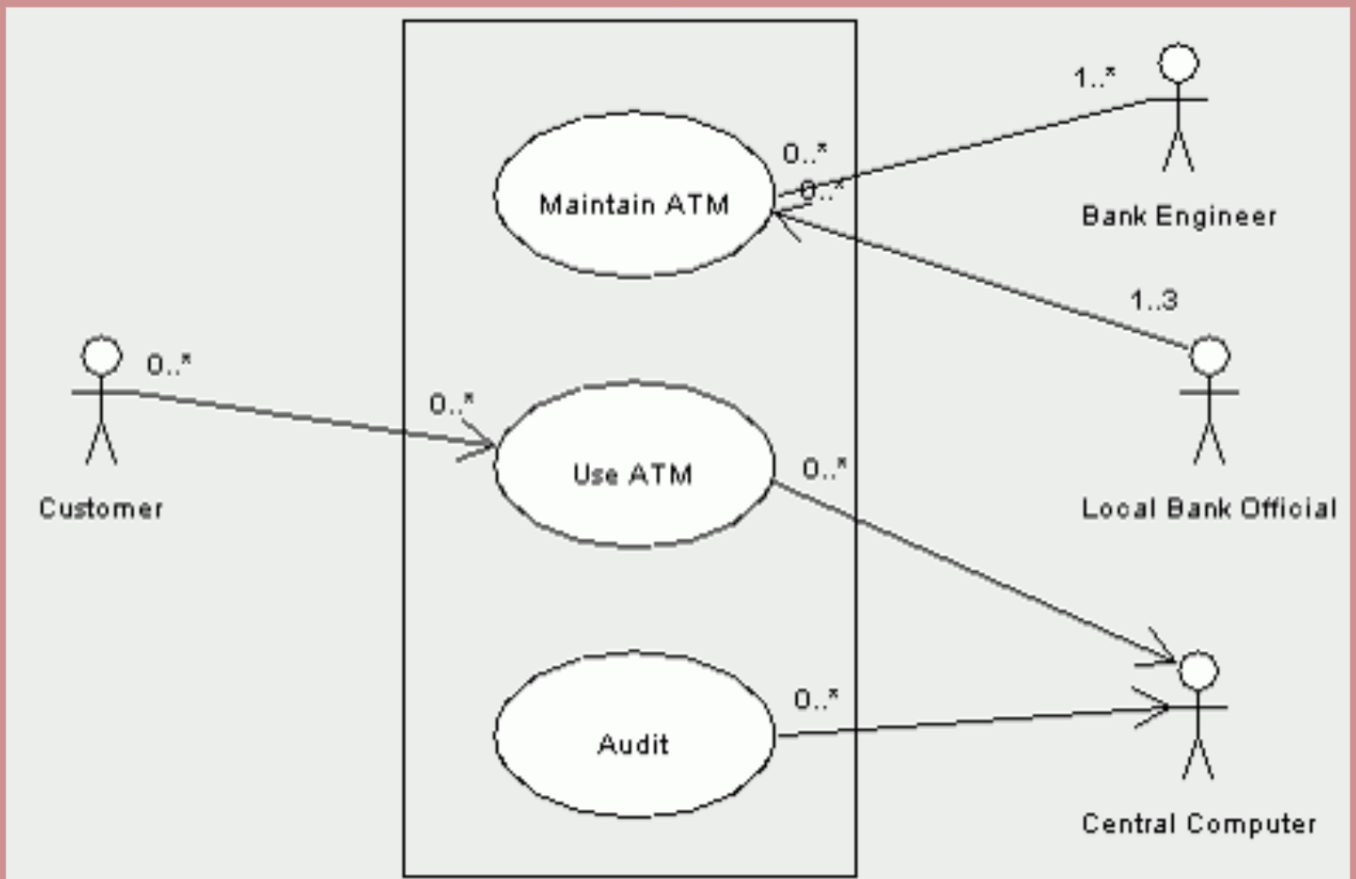
- حالات الاستخدام (البعضاويّات التي تحتوي على الأسماء).
- لكل فاعل، تُحدد الحالات التي يبدأها أو يشارك فيها.
- مثل: تسجيل الدخول، شراء منتج، استرداد مال.

ملاحظة:

- يتم ربط الفاعلين بحالات الاستخدام بخطوط فقط.
- تجنب استخدام الأسهم، لأنها قد تُسبب الغموض في التفسير.

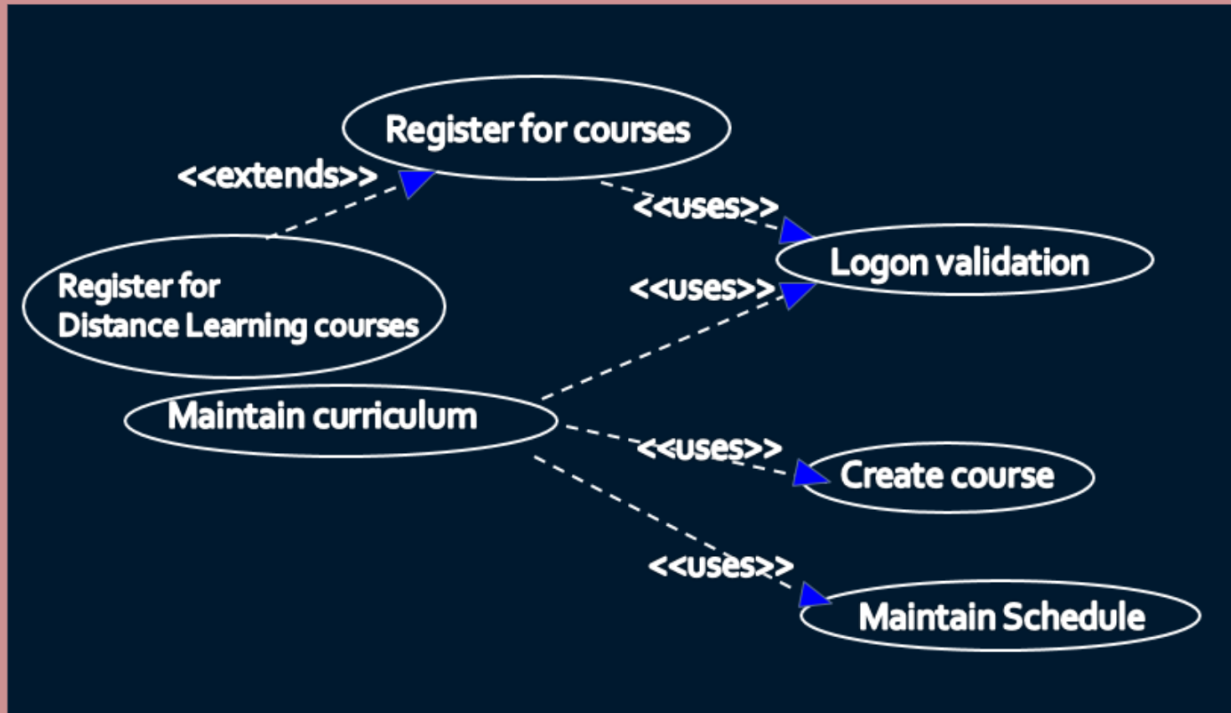
✓ مثال آخر على حالة استخدام عالية المستوى

(High Level Use Case)

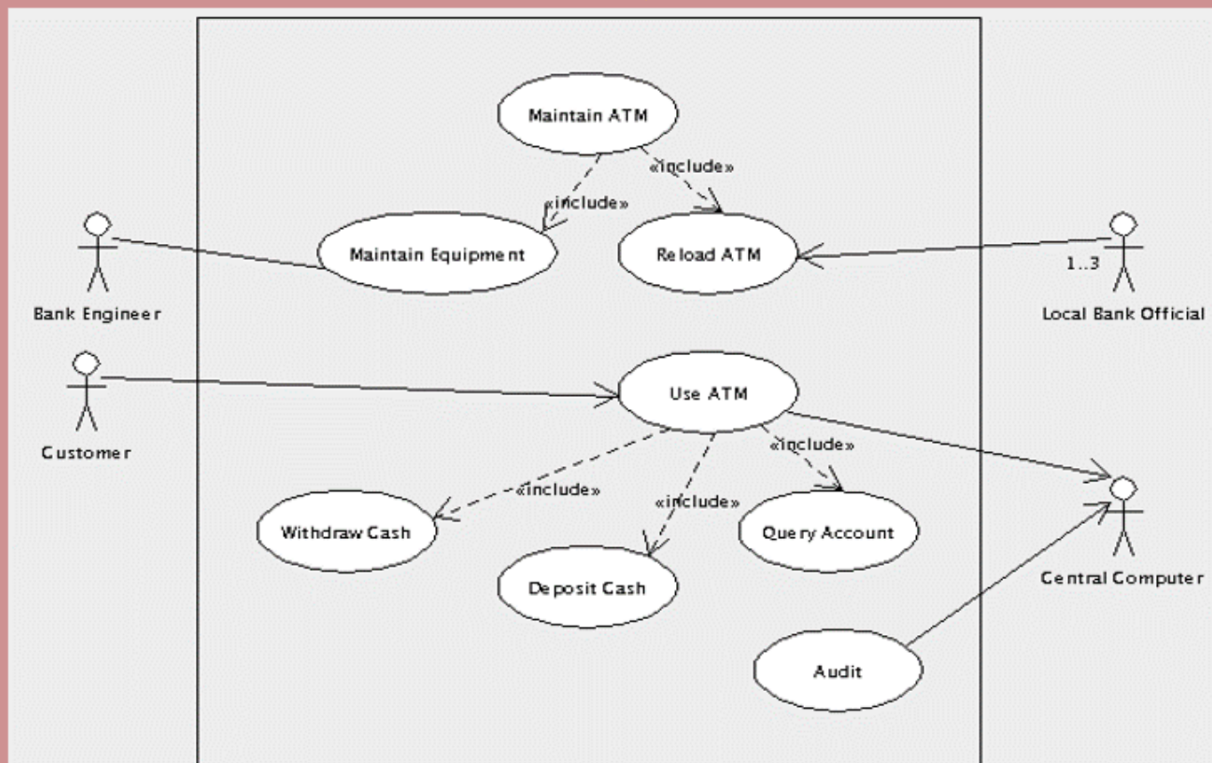




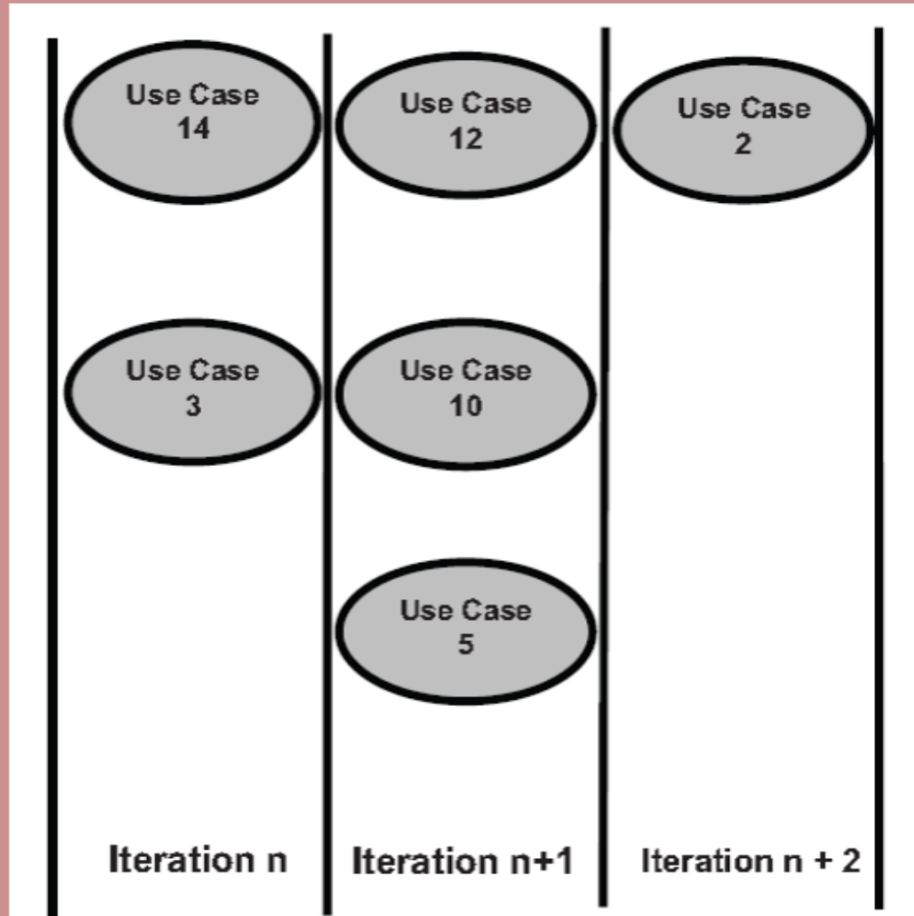
استخدامات (Uses) و الامتدادات (Extends) في
مخطط حالات الاستخدام (Use Case Diagram)
تظهر علاقة "يستخدم" (Uses) سلوكًا مشتركًا بين واحدة
أو أكثر من حالات الاستخدام.
تظهر علاقة "يمتد" (Extends) سلوكًا اختياريًا أو استثنائيًا.



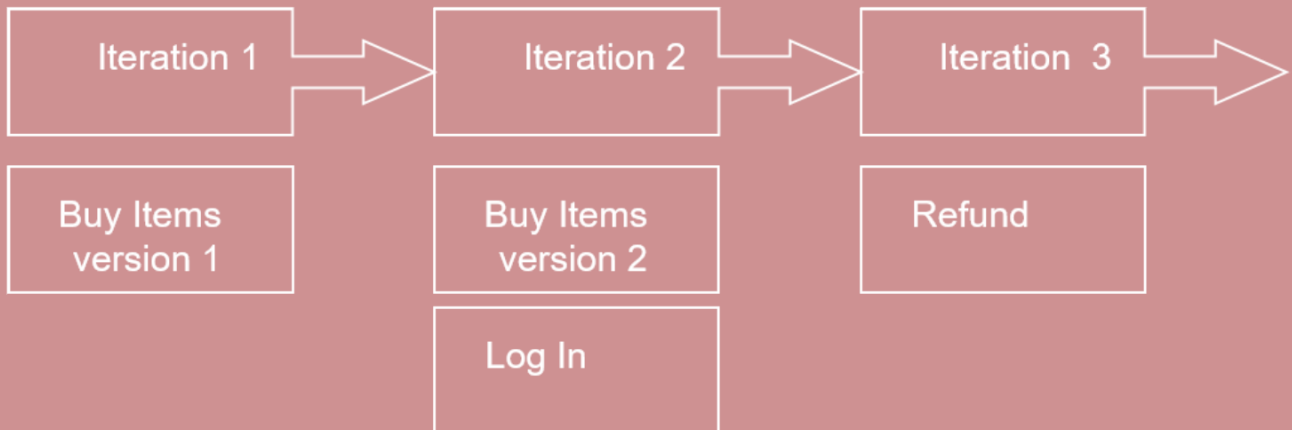
✓ مثال آخر على حالة استخدام موسعة
(Expanded Use Case)



تخصيص حالات الاستخدام للتكرارات 
تستخدم حالات الاستخدام لتخطيط محتوى التكرار.



تخصيص حالات الاستخدام للتكرارات 



حالات الاستخدام وقصص المستخدم 
(Use Cases and User Stories)

أوجه التشابه والاختلاف: 

- وجه الشبه: كلاهما وسيلتان لتنظيم المتطلبات.
- الاختلاف: كل منهما ينظم المتطلبات لأغراض مختلفة.

قصص المستخدم (User Stories)

- عرض بسيط ومباشر للمعلومة.
- تُصاغ بلغة المستخدم اليومية.
- لا تحتوي على تفاصيل كثيرة، مما يترك المجال للتفسير.
- تساعد القارئ على فهم ما يجب أن ينجزه البرنامج.
- يجب أن تُرفق بمعايير قبول (Acceptance Criteria) لتوضيح السلوك المتوقع عندما تكون القصة غامضة.

حالات الاستخدام (Use Cases)

- تنظم المتطلبات لتكوين سرد سردي (قصة) لكيفية تفاعل المستخدمين مع النظام واستخدامه.
- تركز على أهداف المستخدم وكيفية تحقيق هذه الأهداف من خلال تفاعلهم مع النظام.
- تصف العملية وخطواتها بالتفصيل.
- يمكن صياغتها ضمن نموذج رسمي.
- الغرض منها تقديم تفاصيل كافية لفهمها بشكل مستقل.
- يمكن تسليمها في وثيقة مستقلة.

تعرف على أنها:

"وصف عام لمجموعة من التفاعلات بين النظام وواحد أو أكثر من الفاعلين (Actors)، حيث يمكن أن يكون الفاعل مستخدمًا أو نظامًا آخر."

ال XP Stories (قصص في منهجية XP)

- تُسمى أحيانًا "Features".
- تُستخدم لتقسيم المتطلبات إلى أجزاء صغيرة قابلة للتخطيط.
- يتم تفصيلها وتقسيمها حتى يمكن تقديرها ضمن خطة الإصدار (Release Planning) في XP.
- قصص المستخدم (User Stories) أبسط وأكثر مباشرة من حالات الاستخدام (Use Cases).

نموذج المسؤولية والتعاون

(Class Responsibility Collaborator - CRC)

- جزء من النمذجة في منهجية XP.
- يشترك في خصائص عديدة مع مخطط الصنف (Class Diagram).
- يُعد أكثر تفصيلاً، وغالبًا ما يُستخدم قبل إنشاء مخطط الأصناف.

ما هو نموذج CRC؟

عبارة عن بطاقات فهرسة (Index Cards) مقسمة إلى 3 أقسام:

- 1 **الصنف (Class):** مجموعة من الكائنات المتشابهة.
- 2 **المسؤولية (Responsibility):** ما يعرفه أو يفعله الصنف.
- 3 **المتعاون (Collaborator):** أصناف أخرى يتفاعل معها الصنف لتأدية مسؤولياته.

1 **الصنف (Class)**

- يمثل مجموعة من الكائنات المتشابهة.
- الكائن قد يكون:

شخصًا، مكانًا، شيئًا، حدثًا، مفهومًا، شاشة، تقريرًا... إلخ.

- **مثال:** في نظام إدارة الشحن والمخزون، قد تشمل الأصناف:

Inventory Item, Order, Order Item, Customer, Surface Address.

اسم الصنف (Class Name)

- يُكتب في أعلى البطاقة.
- الاسم يبني مفردات التحليل.
- استخدم الأسماء (nouns) كأسماء أصناف.
- يمكن تحويل الأفعال إلى أسماء إذا كانت تحفظ حالة، **مثل:**
"reads card" ⇒ CardReader
- استخدم أسماء قابلة للنطق.
- ابدأ الحروف بأحرف كبيرة (CardReader).
- تجنب الاختصارات الغامضة أو الغير مفهومة.
- لا تستخدم أرقام داخل الأسماء **مثل:** CardReader1

Class Name	
Responsibilities	Collaborators

Student	
Student number Name Address Phone number Enroll in a seminar Drop a seminar Request transcripts	Seminar

Inventory Item	
Item number Name Description Unit Price Give price	

Order	
Order number Date ordered Date shipped Order items Calculate order total Print invoice Cancel	Order Item Customer

Order Item	
Quantity Inventory item Calculate total	Inventory item

Customer	
Name Phone number Customer number Make order Cancel order Make payment	Order Surface Address

Surface Address	
Street City State Zip Print label	

Figure 2. A CRC model for a simple shipping/inventory control system.

2 المسؤولية (Responsibility)

- تمثل ما يعرفه الصنف (Class) أو ما يفعله.
- مثال: الزبون يعرف الاسم، رقم الحساب، رقم الهاتف.
- ويقوم بـ: الطلب، الإلغاء، الدفع.

تصف سلوك الصنف.

استخدم عبارات فعلية قصيرة، مثل:

"reads card"، "look up words"

لا توضح كيفية التنفيذ، فقط ما يجب القيام به.

لماذا استخدام بطاقات محدودة الحجم مفيد؟

- تساعد في تقدير مستوى التعقيد المناسب للصنف.
- إذا لم تستطع تلخيص المهام على بطاقة واحدة، فقد تحتاج إلى تقسيم المهام إلى أصناف متعددة.

3 المتعاونون (Collaborators)

- الأصناف الأخرى التي يتعاون معها الصنف لتحقيق مسؤولياته.
- تشمل الموردّين المهمين أو العملاء المحتملين.
- لماذا نهتمّ بالموردّين؟

لأنهم ضروريون لتحقيق المسؤوليات.

- مثالاً: المسؤولية "read dictionary" تفترض وجود صنف "Dictionary" كمتعاون.

مخطط الأصناف (Class Diagram)

- مفيد في جميع أنماط البرمجة الكائنية (OOP).
- يُستخدم في UML لعرض المحتوى الثابت للأصناف والعلاقات بينها.

يُستخدم لنمذجة:

- الهيكل البياني لمجال معين (Domain Structure)
- التصميم التفصيلي للنظام المستهدف.

مخطط الأصناف (Class Diagram)

- **الصف (Class):** هو وصف لمجموعة من الكائنات (Objects) التي تشترك في نفس السمات (Attributes)، والعمليات (Operations)، والعلاقات (Relationships)، والدلالات (Semantics).

التمثيل الرسومي: 🎨

- يرسم الصف على شكل مستطيل يحتوي عادةً على ثلاثة أقسام: اسم الصف (الجزء العلوي)
 - السمات (المنتصف)
 - العمليات (الأسفل)
- اسم الصف (Class Name): هو العنصر الوحيد المطلوب دائمًا، ويظهر في الجزء العلوي من المستطيل. 📎



السمات (Attributes)

● السمة (Attributes) هي خاصية مسماة للصف تُستخدم لوصف الكائنات (Objects) التي يمثلها.

● تظهر السمات في القسم الثاني من المستطيل.

الصيغة العامة: اسم_السمة : النوع 

أنواع الوصول:

● + عام (Public)

● # محمي (Protected)

● - خاص (Private)

● / مشتقة (Derived)

Person

```
name : String
address : Address
birthdate : Date
ssn : Id
```

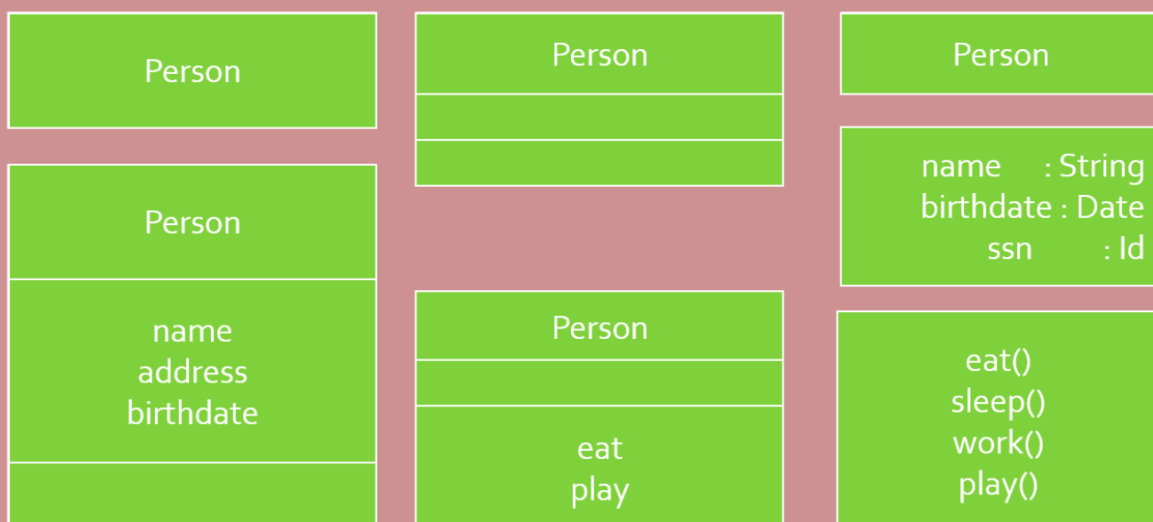
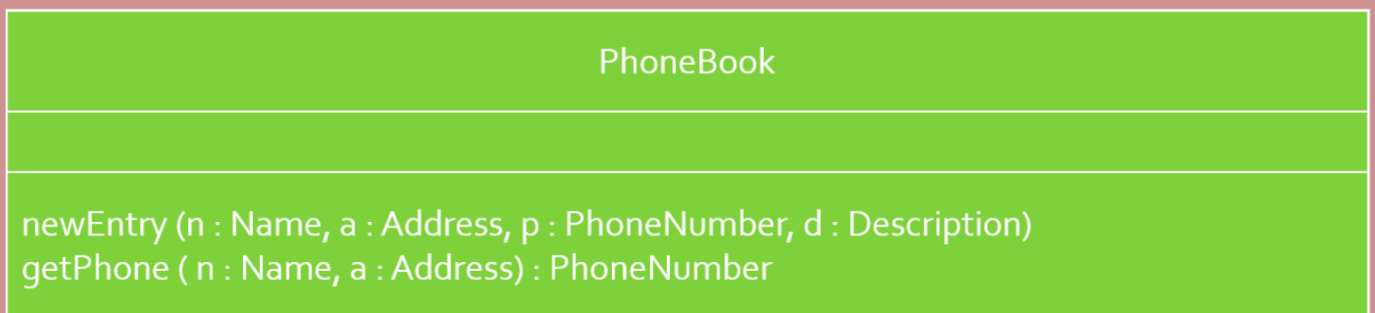
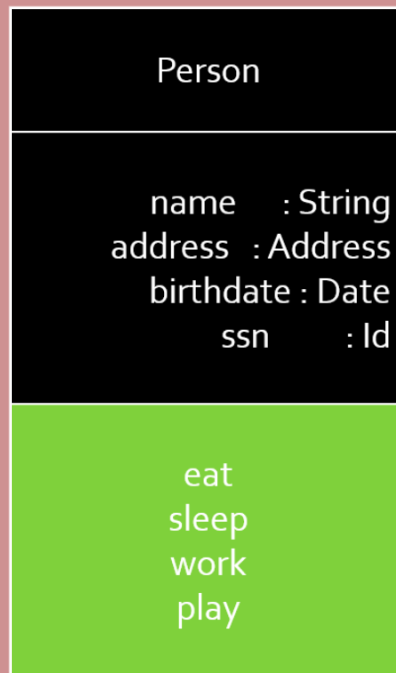
```
+ name : String
# address : Address
# birthdate : Date
/ age : Date
- ssn : Id
```

العمليات (Operations)

- تصف العمليات (Operations) سلوك الصنف وتظهر في القسم الثالث من المستطيل.

صيغة التحديد:

- اسم العملية، أنواع المعاملات، القيم الافتراضية، ونوع القيمة المرجعة (للدوال).
- لا يلزمك إظهار كل السمات والعمليات في كل مخطط صنف. 



العلاقات (Relationships) في UML:

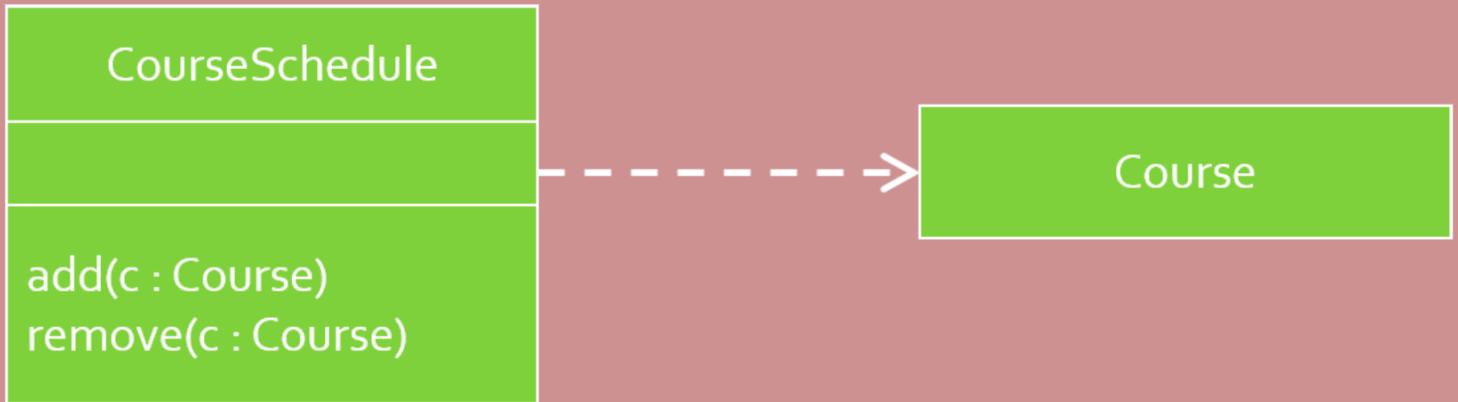
- تُستخدم لتمثيل الاتصالات بين الكائنات.

أنواع العلاقات:

- التبعيات (Dependencies):

علاقة دلالية تعني أن تغييرات في عنصر قد تؤثر على الآخر. 

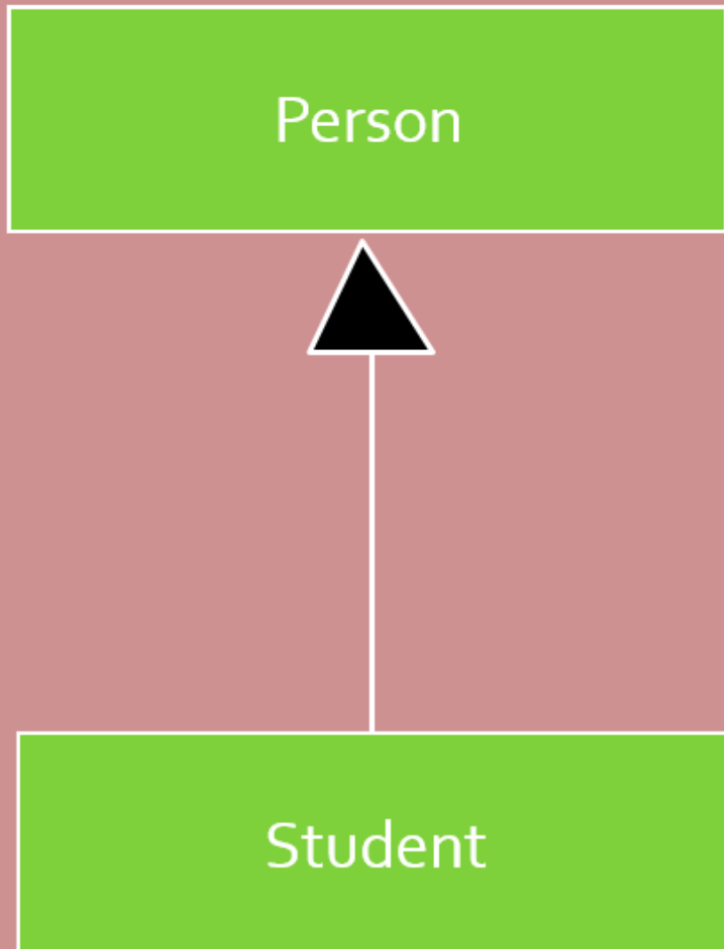
مثال: *CourseSchedule* تعتمد على *Course*.



- التوريث (Generalization):

توصل صنفًا فرعيًا (Subclass) بصنف رئيسي (Superclass), 

ما يعني وراثة السمات (Attributes) والسلوك (Behavior).



• الارتباطات (Associations):

✍ توضح وجود رابط أو تواصل بين صنفين.

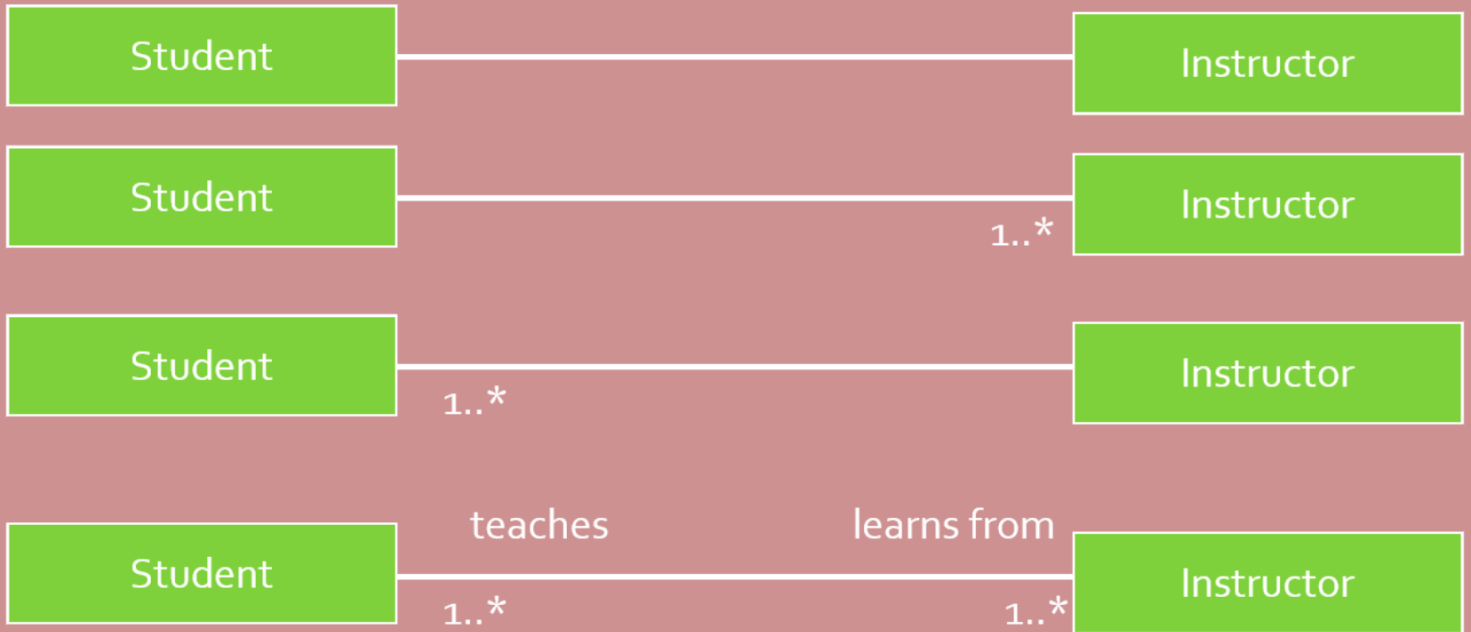
✍ يمكن تحديد العدد (Multiplicity) على كل طرف من الارتباط:

• **مثال:** الطالب له واحد أو أكثر من المعلمين 1..*

كل معلم له واحد أو أكثر من الطلاب 1..*

• الأدوار (Role Names):

توضح وظيفة الكائن (Class) في العلاقة.

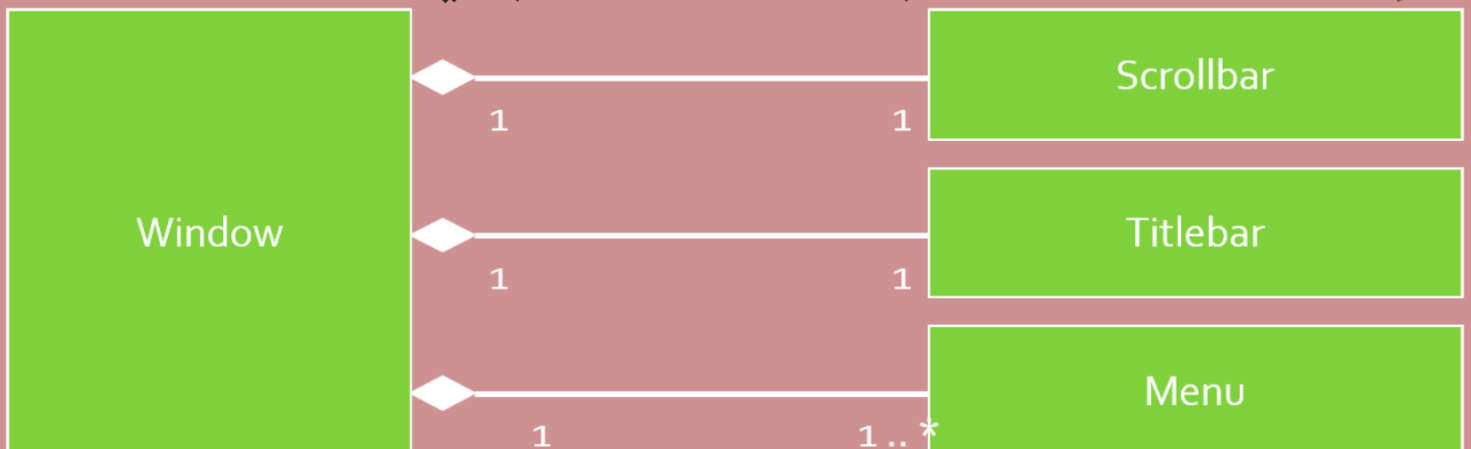


• التركيب (Composition):

✍ يشير إلى ملكية قوية وعمر مشترك بين الكائنات،

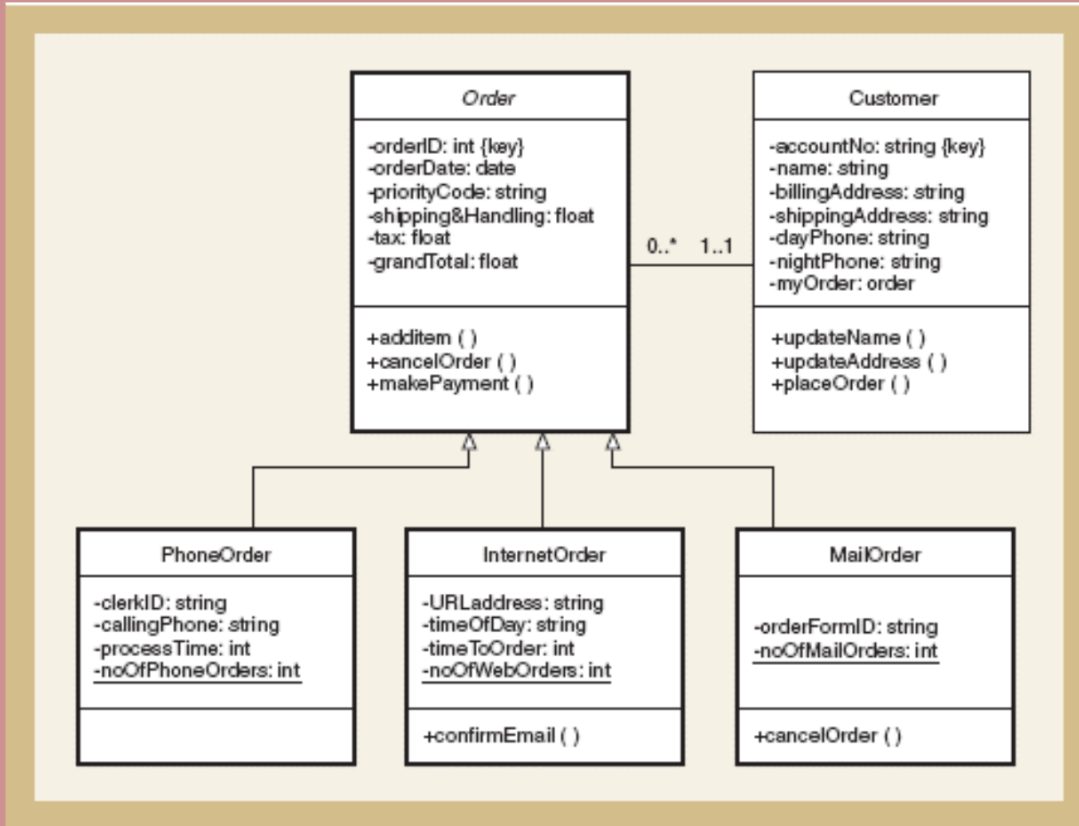
(الجزء لا يمكن أن يوجد بدون الكل).

✍ يُرمز له بـ معين ممتلئ (Filled Diamond) في نهاية العلاقة.



مثال على مخطط صنف

مخطط يحتوي على أصناف تصميم (Design Classes) مع علاقات التوريث.



مخططات التسلسل

(Sequence Diagrams)

توضح كيف تتفاعل الكائنات مع بعضها البعض مع تسلسل زمني للرسائل.

الميزات:

تؤكد على الترتيب الزمني للتفاعلات.

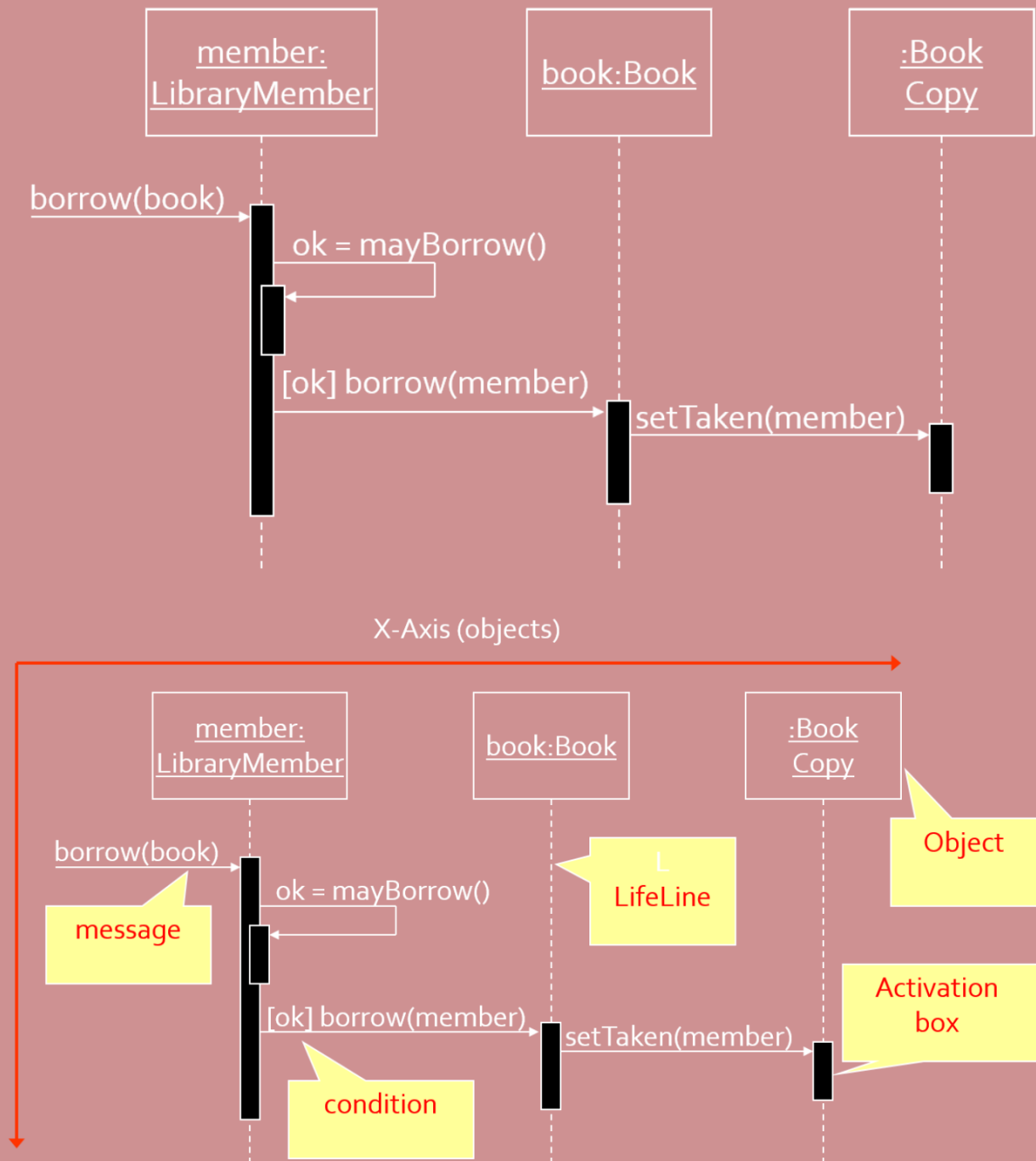
يمكنها تمثيل:

التدفق المتسلسل البسيط, التفرع (Branching), التكرار (Iteration), الاستدعاء الذاتي (Recursion), التزامن (Concurrency)

أهمية مخططات التسلسل في XP:

- تساعد المطورين على فهم التفاعل الزمني بين كائنات النظام.
- تساعد العملاء ذوي الخبرة التقنية على تصور كيفية عمل النظام.
- تساعد في تحديد نقاط الفشل وتسهيل إعداد حالات الاختبار (Test Cases).

مثال على مخططات التسلسل



تفاصيل الرسم:

- الكائنات توضع أفقيًا في الأعلى.
- الزمن يتقدم عموديًا، لذا يُقرأ المخطط من الأعلى إلى الأسفل.
- التفاعلات (الرسائل) تمثلها أسهم مسماة.
- نوع السهم يحدد نوع التفاعل (استدعاء، استجابة... إلخ).
- المستطيل النحيف داخل "خط حياة" الكائن يسمى مربع التفعيل (Activation Box) ويمثل الوقت الذي يكون فيه الكائن هو المتحكم في النظام.

الكائن (Object) 🧱

تسمية الكائنات (Object Naming) 🚫

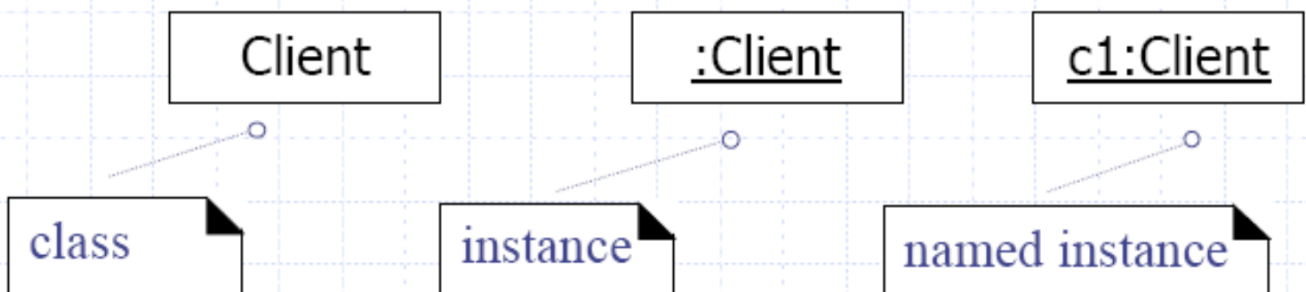
- **الصيغة:** [اسم_المثيل] : [اسم_الصنف]
- استخدم نفس أسماء الأصناف (Classes) كما في مخطط الصنف.
- يجب إدراج اسم المثيل (instance name) عندما:
 - تتم الإشارة إلى الكائن في رسالة.
 - يوجد عدة كائنات من نفس النوع في المخطط.

خط الحياة (Life-Line) 🧬

- يُمثل فترة حياة الكائن أثناء التفاعل.

myBirthdy
:Date

• Classes and Instances



الرسائل (Messages)

تعريف:

التفاعل بين كائنين يُمثّل كرسالة تُرسل من كائن إلى آخر.

قد تكون الرسالة:

- استدعاء دالة
- إشارة (Signaling)
- استدعاء إجراء عن بعد (RPC)
- لإرسال رسالة، يجب أن يكون هناك رابط (Link) بين الكائنين (مثل علاقة تبعية أو انتماء لنفس الكائن).

التمثيل الرسومي:

- يُمثّل الرسالة بسهم بين خطي الحياة للكائنين.
- الاستدعاء الذاتي (Self call) مسموح أيضًا.

مربع التفعيل (Activation Box):

- يرمز إلى المدة الزمنية التي يستغرقها الكائن لمعالجة الرسالة.
- يُرسم كمستطيل أبيض ضيق على خط الحياة.

صيغة الرسالة (Message Notation):

- قيمة_الإرجاع := اسم_الرسالة (اسم_المعطى: نوع_المعطى):
نوع_القيمة_الراجعة

أمثلة: ✓

`s := calculateSalary(hours)`

`s := calculateSalary(hours: Integer): Double`

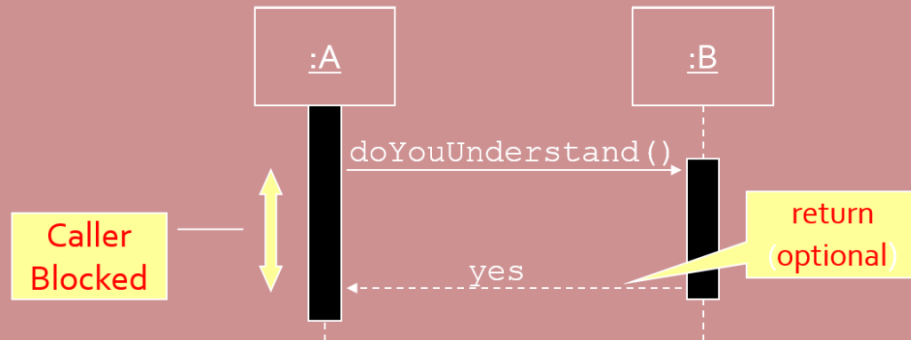
قيم الإرجاع (Return Values):

- يمكن إظهارها اختياريًا عبر سهم متقطع يُظهر اسم القيمة المُرجعة.
- لا تُظهر قيمة الإرجاع إذا كانت واضحة (مثل `getTotal()`).
- أظهرها فقط عند الحاجة لاستخدامها في موضع آخر (مثل تمريرها في رسالة أخرى).
- يُفضل دمج قيمة الإرجاع في استدعاء الدالة: (مثل `ok = isValid()`)



(Synchronous Messages)

- تدفق التحكم متداخل، أي:
- يجب إتمام الرسالة قبل أن يستأنف الكائن المُرسِل التنفيذ.
- ينتظر التنفيذ حتى يتم استلام الرسالة ومعالجتها.



التفعيلات (Activations)



- استدعاء إجراء يُفعل الكائن.
- يُظهر مربع التفعيل الفترة الزمنية التي يكون فيها الكائن نشطًا.

نماذج التنفيذ:

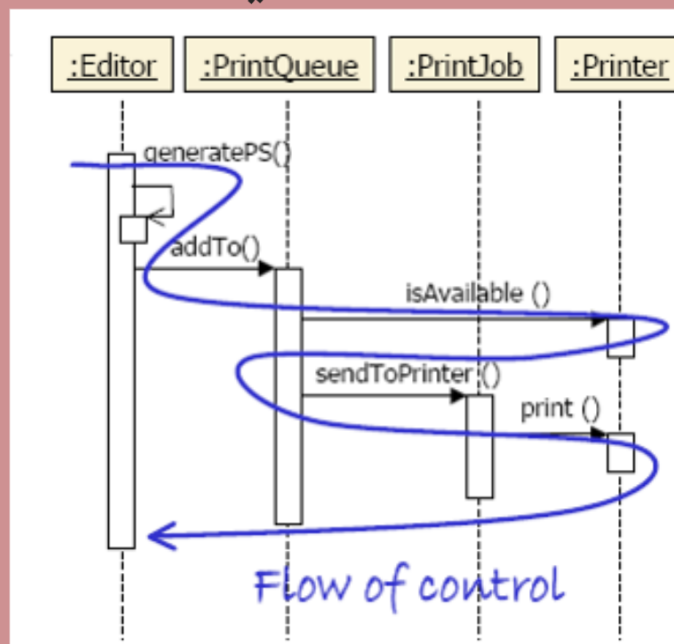


1 نموذج أحادي المسار (Single-thread):

- كائن واحد فقط نشط في لحظة معينة.

2 نموذج متعدد المسارات (Multi-thread):

- يمكن أن تكون عدة كائنات نشطة في نفس الوقت.



- يُعرض التفعيل على شكل صندوق أبيض على خط حياة الكائن. ويظهر متى يكون ذلك الكائن نشطًا.
- يؤدي استلام رسالة إلى تفعيل الكائن، ويظل نشطًا أثناء المعالجة أو أثناء انتظار عودة استدعاءات الإجراءات المتداخلة.
- تعيد الإجراء (Procedure) التحكم ضمنيًا إلى الكائن المستدعي عند نهاية تفعيله.
- وتعد التفعيلات مفيدة في توضيح استدعاءات الإجراءات المتداخلة، حيث يغادر سهم الرسالة مستطيل التفعيل وليس خط الحياة.

المعلومات التحكمية

(Control Information)

الشرط (Condition)

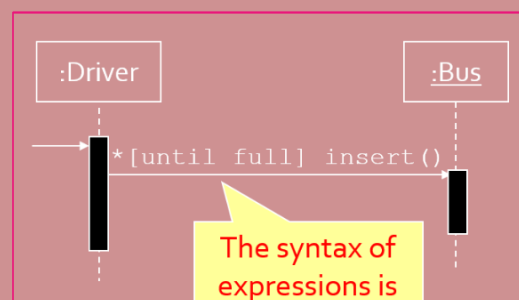
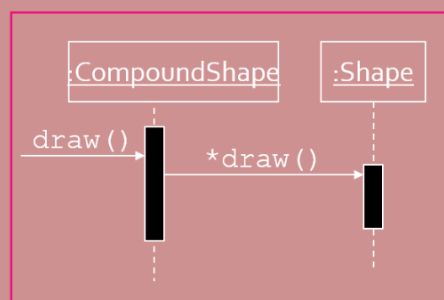
- الصيغة: [شرط] اسم_الرسالة
- ترسل الرسالة فقط إذا تحقق الشرط.
- ✓ مثال:

[ok] borrow(member)



التكرار (Iteration)

- الصيغة: * [شرط] اسم_الرسالة
- تُرسل الرسالة عدة مرات إلى كائن واحد أو عدة كائنات.
- ✓ مثال:



The syntax of expressions is not a standard

ملاحظات مهمة:



- آليات التحكم في مخططات التسلسل كافية لتمثيل البدائل البسيطة فقط.
- استخدم عدة مخططات تسلسل لتمثيل السيناريوهات المعقدة.
- لا تستخدم مخططات التسلسل لتصميم الخوارزميات بالتفصيل.

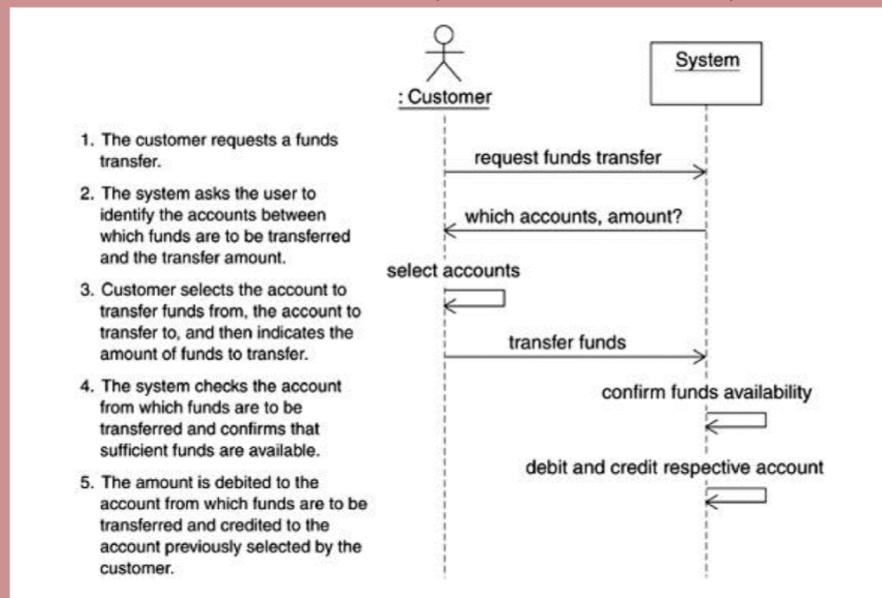
الأفضل استخدام:



مخططات النشاط (Activity Diagrams)

الرمز الزائف (Pseudo-code)

مخططات الحالة (State-charts)



Session Sequence Diagram

